

START HERE

From paper proof to program

A guided route through BFS, insertion sort, and the proof obligations you actually write.

You already know what a loop invariant is. This tutorial shows how to turn that knowledge into a checked correctness and time theorem for a RAM program. **No compiler knowledge is assumed.**

RUNNABLE LEAN

```
import AlgoLib.Experimental.RAM
open AlgoLib.Experimental.RAM.Paper

#eval (Insertion.run [3, 1, 2]).value
-- [1, 2, 3]
```

Your destination

Run an algorithm, explain its invariant and charging argument, and identify which obligations are yours and which the library discharges.

Choose a route

I want to ...	Read
Run an example and understand its output	Pages 2-4
Understand correctness and time credits	Pages 5-7
Read and interact with a real Lean proof	Pages 8-9
See the component map and solve exercises	Pages 10-12

Before you begin

Use a checkout of **experimental-RAM-model** with Lean and Lake available. The repository pins Lean in lean-toolchain; use that version. The companion **StudentDemo.lean** is a complete, checked file. Code excerpts below live inside its namespace unless a box says otherwise.

Labels distinguish **RUNNABLE LEAN**, **REPOSITORY EXCERPT**, and **PSEUDOCODE**. You can understand the paper argument before learning every Lean tactic.

1. GET A FIRST RESULT

Run a graph you can draw

The demo has a diamond-shaped component and one isolated vertex.

RUNNABLE LEAN / INSIDE STUDENTDEMO.LEAN

```
def graph : EdgeInput where
  n := 5
  edges := [(0, 0, 1), (1, 0, 2),
            (2, 1, 3), (3, 2, 3)]
  distinct := by decide
  valid := by decide

#eval (BFS.run (graph.fromSource 0 (by decide))).value.toList
-- [0, 1, 2, 3]

#eval (BFS.run (graph.fromSource 4 (by decide))).value.toList
-- [4]
```

Each triple is **(edge label, endpoint, endpoint)**. Vertex identifiers are 0 through 4. `distinct` checks the finite edge data; `valid` checks that the endpoints are in range. The source proof checks that the chosen source is a vertex.

Run the companion

Copy **StudentDemo.lean** into the repository root. From that directory, run:

SHELL COMMANDS

```
lake build
lake env lean StudentDemo.lean
```

The file uses `#guard_msgs` to check expected outputs. A successful run can therefore be silent. In your editor, select a `#eval` command to inspect its result, or remove its surrounding output assertion while experimenting.

Read the result correctly

value is a visited bitmap; **toList** displays its marked vertices in increasing order. It is not the dequeue order. **steps** counts actual compiled RAM steps. You supply no fuel.

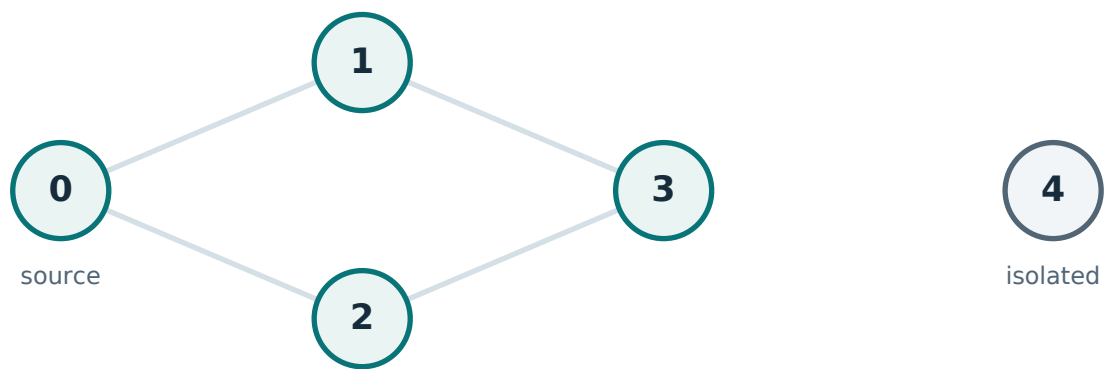
[Get the complete StudentDemo.lean companion](#)

[Open the checked example API](#)

2. MAKE THE STATE CONCRETE

Trace the frontier by hand

At each outer-loop boundary, all neighbors of the processed vertices have been examined.



Use source 0 and list its neighbors as [1, 2]. FIFO means that newly discovered vertices join the back of the queue. Vertex 3 is discovered from vertex 1; scanning vertex 2 must not enqueue it again.

After	processed	queue	seen
Initialization	{}	[0]	{0}
Process 0	{0}	[1, 2]	{0, 1, 2}
Process 1	{0, 1}	[2, 3]	{0, 1, 2, 3}
Process 2	{0, 1, 2}	[3]	{0, 1, 2, 3}
Process 3	{0, 1, 2, 3}	[]	{0, 1, 2, 3}

Check your understanding

Why is 3 present only once in the queue? Because discovery marks it **before** enqueueing it. The later edge from 2 sees that mark. Vertex 4 is never discovered.

The displayed state is mathematical. The library represents it with visited flags, a FIFO, and an adjacency cursor in RAM. You reason about the table; the library proves that the representation follows it.

3. CONNECT PSEUDOCODE TO THE API

Read the program in layers

The current interface uses certified operations. Some textbook lines are grouped into one library call.

PSEUDOCODE / FOR READING, NOT LEAN INPUT

```
clear all visited flags; mark and enqueue source
while Q is not empty:
  u := dequeue(Q)
  for v in adjacency[u]:
    if not visited[v]:
      visited[v] := true
      enqueue(Q, v)
```

REPOSITORY EXCERPT / PAPER/BFS.LEAN

```
def program (a : Adjacency) : Program (model a) := paper {
  while (queueNonempty a) {
    call dequeue a;
    call (scanNeighbors a).call;
    call finish a;
  }
}
```

Component	Meaning you use
Input preparation	Clears flags, marks the source, and seeds the FIFO. This is compiled and charged.
dequeue	Removes the front vertex and opens that vertex's adjacency row.
scanNeighbors	Scans the row; marks and enqueues previously unseen vertices. Its loop has a reusable proof.
finish	Records the current vertex in the mathematical processed set. It emits no instructions.

What “call” promises: if you establish the precondition, you may use the operation's mathematical effect and cost bound. Its implementation proof already includes the physical memory frame.

[Read the row procedure and its contracts](#)

4. STATE THE ALGORITHMIC REASON

Write the invariant in English

An invariant describes every loop boundary, not just the final answer.

Invariant fact	Why the proof needs it
seen = processed $\cup$ queued	Every discovered vertex is either finished or still waiting.
No duplicates; processed and queue are disjoint	Each vertex is processed at most once.
The source is seen; every seen vertex is reachable	No unrelated vertex enters the answer.
All neighbors of processed vertices are seen	When the queue empties, no reachable vertex can have been missed.
No partially scanned row at the outer header	Each outer iteration completely finishes one vertex.

The paper argument

Initialization. Only the source is seen and queued; processed is empty. Reachability of the source follows from the length-zero path. Closure for processed vertices is vacuous.

Preservation. Dequeue a reachable vertex u . Every newly discovered neighbor is reachable via u . Mark-before-enqueue preserves uniqueness. After scanning the whole row, u may safely join processed.

Exit. The queue is empty, so seen = processed. The set contains the source and is closed under following edges. Induction along any path from the source shows that every reachable vertex is seen. The soundness part gives the reverse direction.

Two different conclusions

Reachability: marked(v) iff v is reachable from the source, even on a disconnected graph.

Connectivity: every graph vertex is marked iff the graph is connected, given a valid source.

[Read the packaged graph invariant](#)

5. MEET SYMBOLIC EXECUTION

See what a proof goal asks

A verification condition is a list of facts needed to justify the next command and the desired result.

For an operation with precondition P , effect E , and cost c , symbolic execution asks you to prove: **P holds; c fits the budget; the desired postcondition holds after E .** It substitutes the logical effect instead of exposing RAM instructions.

RUNNABLE LEAN / LOCAL CONTRACT DEMO

```
example (s : Insertion.State) (safe : s.todo ≠ []) :
  VC (.action Insertion.insertNext)
    (fun t _ => t = Insertion.effect s)
    s (s.sorted.length + 1) := by
  paper_steps []
  exact (safe, Nat.le_refl _, by trivial)
```

What you see	How to read it
<code>safe : s.todo ≠ []</code>	The insertion operation has a value available to insert.
<code>fun t _ => ...</code>	The desired postcondition describes the new state <code>t</code> . The underscore ignores leftover credits.
<code>s.sorted.length + 1</code>	The supplied budget, measured in the library's work units.
<code>paper_steps []</code>	Use registered operation contracts to generate logical obligations.
<code>Nat.le_refl _</code>	The supplied budget equals the required cost, so it fits.

Try it in the editor

Put the cursor after `paper_steps []` and inspect the goals. Temporarily replace the budget by 0. The proof should fail because a nonempty insertion costs positive work. Restore the original budget afterward.

This is a warm-up about one operation. A loop additionally needs an invariant, a potential, and an exit argument. You supply those mathematical facts; the framework propagates the contracts.

6. PROVE A LINEAR BOUND

Pay once for each vertex

Credits are a proof budget. They do not exist in the executed data structure.

Let d be the length of the dequeued vertex's adjacency row. The public contracts allocate the following work units for one outer iteration:

Work	Credits
True outer-loop guard	1
dequeue and open the row	1
scanNeighbors: d visits and $d + 1$ row tests	$2d + 1$
finish: ghost bookkeeping	0
Total per processed vertex	$3 + 2d$

Choose a potential

$\Phi = \text{sum of } (3 + 2 \text{ degree}(v)) \text{ over all unprocessed vertices.}$

A fresh vertex leaves this sum when it is processed. The removed summand pays for its whole iteration.

On the diamond-plus-isolated-vertex demo, the four diamond vertices each have degree 2. They receive 7 credits each; vertex 4 receives 3. The potential follows $31 \rightarrow 24 \rightarrow 17 \rightarrow 10 \rightarrow 3$. The unspent 3 belongs to the unreachable vertex. Reserve one extra credit for the final false outer guard.

In general, the initial potential is $3n + 2A$, where A is the number of stored adjacency entries. For an undirected graph, $A \leq 2m$. Thus the loop uses at most $3n + 4m + 1$ work units. Certified input preparation also takes linear work.

The executable theorem

BFS.linear input proves $\text{steps} \leq 370(n + m)$ for actual compiled RAM instructions, including initialization. The framework handles the conversion from library work units. You do not derive the compiler constant.

The bound is conservative, not an exact runtime. Here $n = 5$ and $m = 4$, so it gives $\text{steps} \leq 3330$. Host input encoding and output formatting are outside the RAM count; this is unit-cost RAM complexity, not bit complexity.

7. A SECOND INVARIANT

Use insertion sort to practise

This implementation grows a sorted suffix from right to left.

REPOSITORY EXCERPT / PAPER/INSERTIONSORT.LEAN

```
def program : Program model := paper {
  while (more) { call insertNext; }
}

def invariant (input : List Nat) (s : State) : Prop :=
  s.sorted.Pairwise ( $\cdot \leq \cdot$ )  $\wedge$  (s.todo ++ s.sorted).Perm input

def potential (s : State) : Nat :=
  s.todo.length * (s.todo.length + s.sorted.length + 2)
```

todo lists the unprocessed prefix in processing order, so it starts as the reverse of the input. **sorted** is the sorted suffix. `insertNext` inserts the head of `todo` into that suffix and removes it from `todo`.

Input [3, 1, 2]	todo	sorted	Potential
Initially	[2, 1, 3]	[]	15
Insert 2	[1, 3]	[2]	10
Insert 1	[3]	[1, 2]	5
Insert 3	[]	[1, 2, 3]	0

Pairwise ($\cdot \leq \cdot$) means that earlier elements are no greater than later ones. **Perm** means that the same values occur with the same multiplicities. Both matter: a sorted output that loses a duplicate is not a correct sort.

Each iteration removes one `todo` value, preserves the total length, and uses at most one suffix scan plus guard overhead. The potential drops by $n + 2$, enough to pay. At exit `todo` is empty, so the sorted suffix is a permutation of the entire input.

What is supplied here

The library already proves the insertion operation's mathematical effect and linear cost. This exercise teaches how to verify a client loop. Implementing the insertion primitive itself is a separate library task.

8. INTERACT WITH THE THREE OBLIGATIONS

Read a complete loop proof

Open Paper/InsertionSort.lean alongside this page. The proof is complete; the boxes below explain its structure.

PROOF OUTLINE / NOT A COMPLETE LEAN PROOF

```
theorem loopProof (input : List Nat) : LoopProof
  more (.action insertNext) (invariant input) potential
  (fun s => s.todo = [] ∧
    s.sorted.Pairwise (· ≤ ·) ∧ s.sorted.Perm input) where
  preservation := by
    -- Use the insertion contract and list lemmas.
    -- Show the invariant and remaining budget at the new state.
  payment := by
    -- A true guard has at least one available credit.
  exit := by
    -- Empty todo makes the invariant imply the result.
```

Preservation. The real proof splits todo into empty or $x :: xs$. The true guard excludes the empty case. It uses a sorted-insertion lemma, a permutation lemma, and `paper_steps`. Credit arithmetic is discharged by `paper_credits`.

Payment. A nonempty todo gives a positive potential. Paying for the true guard ensures strict progress in the remaining budget, so the same argument establishes termination.

RUNNABLE LEAN / THE EXIT ARGUMENT

```
example (xs : List Nat) (s : Insertion.State)
  (h : Insertion.invariant xs s) (empty : s.todo = []) :
  s.sorted.Pairwise (· ≤ ·) ∧ s.sorted.Perm xs := by
  simpa [Insertion.invariant, empty] using h
```

Read this as: “Expand the invariant, substitute that todo is empty, and simplify.” This is the paper-level exit argument written in Lean. The completed `loopProof.correct` theorem supplies total correctness; the executable interface combines it with initialization and runs the compiler.

Keep the important boundary

You must still prove that the prepared input satisfies the invariant. The library handles memory preparation and its cost; it cannot decide that your chosen invariant is initially true.

[Open the full, checked loop proof](#)

9. KEEP THIS MAP BESIDE YOUR EDITOR

How the components connect

Read the left labels as responsibilities, and the arrows as guarantees passed to the next component.

YOU**Program + invariant + potential**

Describe the algorithm, what stays true, and how work is paid.

**LIBRARY****Operation and procedure contracts**

Provide logical effects, preconditions, cost bounds, and frames.

**AUTOMATIC****Verification conditions**

Substitute effects and collect obligations; you prove the remaining math.

**AUTOMATIC****Checked source and compiler**

Turn the verified composition into typed source, then RAM instructions.

**AUTOMATIC****Input/output interface and runner**

Include paid initialization, run without fuel, and expose the result.

**YOU USE****Correctness + time theorem**

Apply `run_correct`, `connected_iff`, or the time bound to the same run.

Follow one write. A visit changes a flag and perhaps a queue slot. Its certified contract preserves the graph representation. Your proof sees a set/list update. The VCG uses that update; the compiler theorem connects the resulting proof to the actual writes. You never prove adjacency-pointer preservation in the algorithm proof.

10. FIND THE RIGHT KIND OF PROBLEM

When a proof stops working

Use the goal to decide whether a mathematical fact is missing or a routine step needs help.

Symptom	What to do next
"unknown identifier" or unknown tactic	Check the imports and namespace. Start from the complete companion file, not an isolated excerpt.
A precondition remains after <code>paper_steps</code>	Check the operation's logical requirements: <code>nonempty queue</code> , <code>nonempty todo</code> , or <code>valid neighbor</code> . Derive that fact from your invariant.
A credit inequality remains	First supply the charging fact, such as a fresh vertex leaving the potential. Then use <code>paper_credits</code> . A smaller budget may simply be false.
Sortedness or reachability remains	This is algorithm mathematics. Use a list/graph lemma or strengthen the invariant. Compiler tactics cannot supply the missing argument.
A long conjunction appears	Read each conjunct as a precondition, payment, or postcondition. The current interface still exposes some Lean bookkeeping.
Module docstring warning	Place a <code>/-! ... -/</code> module comment immediately after the imports. Do not suppress a real warning as an expected test result.

Three exercises

A. Change the source. Before running source 4, predict `seen`, `queue`, and the final output. Does a correct BFS mark every vertex?

B. Break a budget. In the local contract demo on page 6, replace the supplied budget by 0. Which obligation should become impossible? Restore the file.

C. Keep a duplicate. Run insertion sort on `[3, 1, 3, 2]`. What must the output be? Which part of the invariant rules out silently dropping one 3?

A useful learning sequence

First run an existing program. Next prove a local contract. Then complete an exit proof. Only then attempt a whole loop invariant. The framework currently assumes basic Lean proof skills; the companion and these exercises provide a starting scaffold.

11. CONTINUE WITHOUT GETTING LOST

Answers and a reading map

Use the smallest entry point that answers your current question.

Exercise answers

- A.** Initially $\text{seen} = \{4\}$, $\text{queue} = [4]$, $\text{processed} = \{\}$. Vertex 4 has no neighbors. Processing it leaves queue empty and the output $[4]$. BFS is correct; the graph is disconnected.
- B.** The operation requires $\text{sorted.length} + 1$ credits, which is always positive. The generated payment obligation cannot fit that amount into a zero budget.
- C.** The output is $[1, 2, 3, 3]$. The permutation invariant preserves multiplicities. Sortedness alone would not rule out $[1, 2, 3]$.

Clickable repository map

[Run and apply theorems](#)

Minimal examples with ordinary inputs and named results.

[BFS program and invariant proof](#)

The frontier argument and per-vertex charging scheme.

[Insertion-sort proof](#)

A small loop with a sortedness/permutation invariant.

[Public graph-operation contracts](#)

What calls require, change, and cost.

[Tutorial and API notes](#)

More detail on syntax, goals, and current limitations.

[Architecture map](#)

Optional: how maintainers connect the proof and execution layers.

Five words to remember

Invariant: a fact true at every loop header. **Potential:** a budget attached to a state. **Contract:** what a call requires and guarantees. **Frame:** what a call preserves. **VC:** an obligation sufficient to justify the next program fragment.

This guide is tied to the checked repository baseline **09418ad**. The examples use the unit-cost natural-number RAM model. New data-structure implementations require library verification; general ownership inference and automatic invariant discovery are not provided.